

REPORTE TAREA 1

ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Martín Araya Díaz

11 de septiembre de 2026

18:14

1. Introducción

En este informe se estudian dos problemas: el ordenamiento de arreglos unidimensionales y la multiplicación de matrices cuadradas. Para el problema de ordenamiento se analizan experimentalmente cuatro algoritmos de ordenamiento (*std::sort*, *PatienceSort*, *QuickSort* y *MergeSort*) y para el de matrices se analizan dos algoritmos (*Naive* y *Strassen*). El objetivo principal es analizar el comportamiento práctico tanto para tiempo de ejecución (ms) como para consumo de memoria (KB), pudiendo identificar y discutir las diferencias que pueden presentarse con lo dictado en la teoría. Para lograr este objetivo se utilizan casos de prueba generados aleatoriamente, con diferentes tamaños y elementos, para luego visualizar mediante gráficos los resultados cuando estas variables cambian.

2. Entorno experimental

Para que se pueda replicar lo presente en este informe, las características de hardware utilizadas para el desarrollo del mismo son: Procesador: AMD Ryzen 5 5600H; RAM: 16 GB 3200 MT/s; GPU: NVIDIA GeForce GTX 1650; SO: Windows 11 Pro versión 25H2. Se utilizó un subsistema de Linux mediante Ubuntu versión 24.04.4 LTS; Compilador: g++ (Ubuntu 13.3.0-6ubuntu2 24.04.1) 13.3.0

3. Resultados y Análisis

En este caso, se generaron arreglos con datos aleatorios de tamaños 10^1 , 10^3 y 10^5 , donde el dominio de los datos puede ser: $D1, D7$, donde $D1$ quiere decir que cada valor del arreglo es un entero entre 0 y

9, para D7 cada valor va entre 0 y 10^7 . Para las matrices se generaron aleatoriamente con tamaños 2^4 , 2^6 y 2^8 , donde el dominio de los datos puede ser: D0, D10, donde en D0 los valores son binarios: 0,1 y en D10 son enteros entre 0 y 9. El tiempo se midió con `std::chrono::high_resolution_clock` y la memoria mediante el incremento del pico de memoria residente (VmHWM en `/proc/self/status`) antes y después de cada ejecución.

3.1. Ordenamiento de arreglos unidimensionales

Antes de comenzar el análisis, es importante conocer la complejidad temporal y espacial teórica de los algoritmos implementados. Además, cabe aclarar que se cambió la implementación de *QuickSort*, haciendo que el pivote fuera elegido aleatoriamente.

Algoritmo	C. temporal teórica	C. temporal impl.	C. espacial	Fuente
<i>MergeSort</i>	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(n)$	GeeksforGeeks
<i>QuickSort</i>	$O(n \log n)$ prom.	$O(n \log n)$ prom.	$O(\log n)$	GeeksforGeeks (modificado)
<i>PatienceSort</i>	$O(n \log n)$	$O(n^2)$	$O(n)$	GeeksforGeeks
<i>std::sort</i>	$O(n \log n)$	$O(n \log n)$	$O(\log n)$	cppreference

A continuación, el análisis de gráficos:

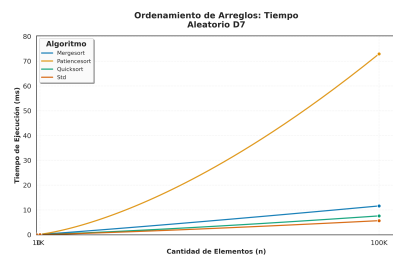


Figura 1: Tiempo de ejecución, arreglos aleatorios (D7).

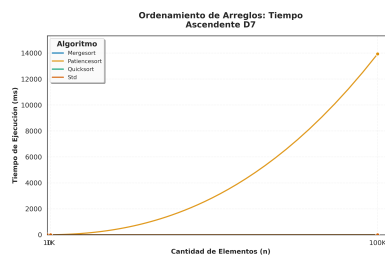


Figura 2: Tiempo de ejecución, arreglos ascendentes (D7).

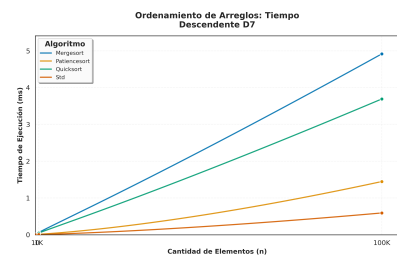


Figura 3: Tiempo de ejecución, arreglos descendentes (D7).

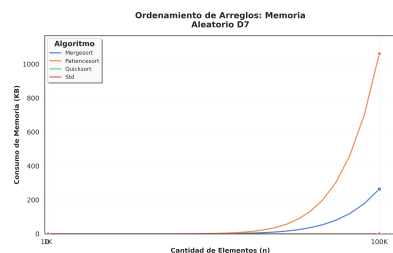


Figura 4: Consumo de memoria, arreglos aleatorios (D7).

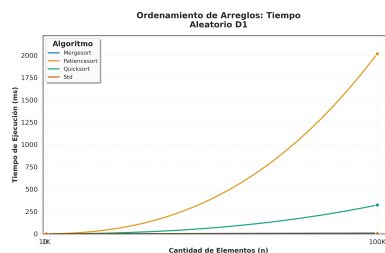


Figura 5: Tiempo de ejecución, arreglos aleatorios (D1).

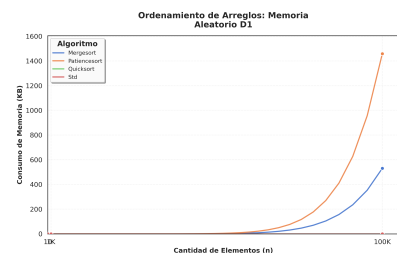


Figura 6: Consumo de memoria, arreglos aleatorios (D1).

En la Figura 1, el orden fue *std::sort* (≈ 6 ms) < *QuickSort* (≈ 8 ms) < *MergeSort* (≈ 12 ms) < *PatienceSort* (≈ 80 ms). Los tres primeros comparten $O(n \log n)$, por lo tanto la diferencia radica simplemente en el valor de la constante que multiplica la complejidad, la cual está dada por una mayor

cantidad de operaciones elementales. Por otro lado, podemos notar claramente el rápido crecimiento de *PatienceSort*.

En la Figura 2 pasa algo interesante, se puede ver claramente lo que sucede en el peor caso de un algoritmo con complejidad $O(n^2)$. Esto se debe principalmente a que se debe crear una pila por elemento y realizar búsquedas lineales constantemente.

En la Figura 3 podemos notar que este caso es mucho más favorable para *PatienceSort*, debido a que cada elemento es mayor o igual que el siguiente que se prueba, utilizando solo una pila y superando a *QuickSort* y *MergeSort*. Esto nos enseña que no hay un caso general para medir el tiempo de un algoritmo con complejidad temporal diferente de $\Theta(f(n))$.

En la Figura 4 se puede observar que el consumo de memoria es coherente con la complejidad teórica, donde *MergeSort* y *PatienceSort* crecen de forma proporcional a $O(n)$, el primero por los vectores temporales que va generando en las recursiones y el segundo debido a las pilas que va generando. Por otro lado *QuickSort* y *std::sort* prácticamente no usan memoria, esto se debe a que teóricamente *std::sort* es in-place y *QuickSort* en teoría no es in-place si se implementa de manera recursiva, porque en cada nivel de la recursión apila el marco de llamadas en el stack referente a los punteros, por lo que el uso de memoria debería crecer con la profundidad ($O(\log n)$ con pivote aleatorio), pero debido a la forma en la que se mide la memoria con VmHWM y a que para $n = 10^7$ consume menos de 4KB, no se detecta prácticamente nada.

En la Figura 5 podemos observar que el tiempo de *PatienceSort* aumenta drásticamente comparado a lo que sucedía en D7. Esto se debe principalmente a que como existen solo diez valores posibles, los arreglos más grandes tienen muchos elementos duplicados y como la condición de *PatienceSort* es buscar elementos estrictamente menores, hace que los duplicados no se puedan almacenar en la misma pila, lo que obliga a crear una pila nueva constantemente y realizar muchas búsquedas lineales.

En la Figura 6 podemos notar la consecuencia directa de lo mencionado anteriormente, *PatienceSort* utiliza una mayor cantidad de memoria debido a la necesidad de manejar con pilas los elementos duplicados, *MergeSort* mantiene su curva y *std::sort* consume ligeramente más memoria que en D7 debido a la existencia de los elementos duplicados, lo que produce una recursión más profunda.

3.2. Multiplicación de matrices cuadradas

Al igual que para los algoritmos de ordenamiento, en este caso es importante conocer las complejidades teóricas y prácticas de los algoritmos de multiplicación de matrices *Naive* y *Strassen*.

Algoritmo	C. temporal teórica	C. temporal impl.	C. espacial	Fuente
Naive	$O(N^3)$	$O(N^3)$	$O(N^2)$	Studymite
Strassen	$O(N^{2,81})$	$O(N^{2,81})$	$O(N^2)$	GitHub (psakoglou)

A continuación, el análisis de gráficos:

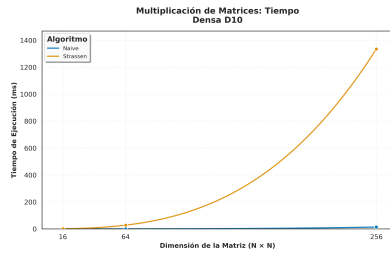


Figura 7: Tiempo, matrices densas (D10).

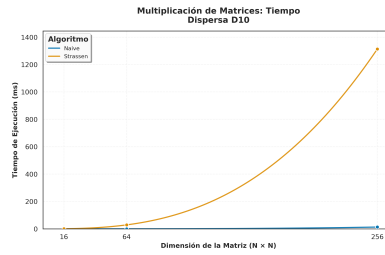


Figura 8: Tiempo, matrices dispersas (D10).

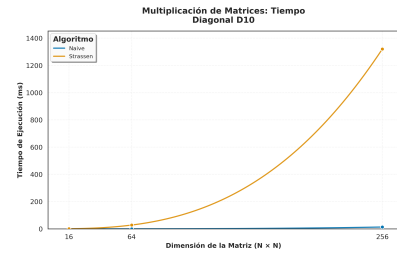


Figura 9: Tiempo, matrices diagonales (D10).

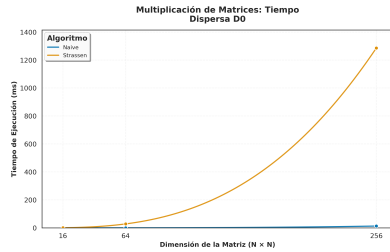


Figura 10: Tiempo, matrices dispersas (D0).

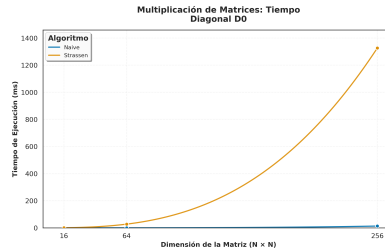


Figura 11: Tiempo, matrices diagonales (D0).

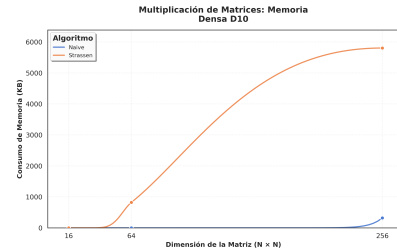


Figura 12: Memoria, matrices densas (D10).

En las figuras 7, 8, 9, 10 y 11 se puede notar que *Strassen* se demora muchísimo más que *Naive*. Si analizamos la función de recurrencia de *Strassen* y usamos el teorema maestro se tiene: $T(N) = 7T(N/2) + \Theta(N^2)$, donde $a = 7$, $b = 2$, $d = 2$ y como $\log_2 7 \approx 2,81 > 2$ estamos en el caso donde $d < \log_b a$ resultando que $T(N) \in \Theta(N^{2,81})$. La teoría nos indica que *Strassen* debería ser más rápido, sin embargo esto no es así en este caso debido a que las constantes involucradas (debido a las operaciones elementales resultantes de generar matrices temporales en cada nivel de recursión hasta llegar al caso base) son tan grandes que se necesita un n extremadamente grande para que este algoritmo sea más rápido que *Naive*. Es un buen ejemplo de cómo se contrasta la teoría con la práctica.

En la figura 12 se puede observar que *Strassen* presenta un consumo muy superior a *Naive*, esto se debe a la memoria auxiliar que se usa debido a las matrices temporales en cada nivel de recursión, mientras que *Naive* solo requiere la matriz de salida.

4. Conclusiones

En conclusión, los diferentes tipos de algoritmos utilizados pueden tener complejidades teóricamente parecidas como *std::sort*, *MergeSort* o *QuickSort*, pero en la práctica podemos notar que existen variaciones temporales (figura 1) o espaciales (figura 6) que nos indican que el análisis de la situación es importante para no elegir algoritmos subóptimos. Un ejemplo de esto se observa en las figuras 2 y 3, donde *PatienceSort* pasa de ser malo en arreglos ascendentes a óptimo en arreglos descendentes. Por otro lado, el mayor contraste entre la teoría y la realidad se puede observar con los algoritmos de multiplicación de matrices, donde *Strassen* parece temporalmente superior a *Naive*, en cambio la práctica nos demuestra lo contrario (figuras 7, 8, 9, 10 y 11) para los casos de prueba utilizados y nos esclarece las consecuencias que tiene ignorar conceptos que en la teoría se omiten.

5. Implementación

El repositorio donde está el código es: https://github.com/Maardiaz/Tarea1_algoco